

Part 1

任务 A

1. ◦ 测量 Model Loading Time
- 仅测量模型资产的加载时间：**起点**设在 `llmrun_support.h` 中 `llm_runtime_init` 函数的 `load_model_cfg(&rt->cfg);` 语句之前，**终点** 设在 `llm_runtime_init` 函数 `memset(rt->seq, 0, sizeof(uint32) * (uint64)rt->cfg.runtime_seq_len);` 语句之后
- 测量端到端初始化时间：把**起点**设在 `llmrun_smol.c` 中 `session_init` 函数的第一条语句之前，**终点**设在 `session_init` 函数的最后一条语句之后
- 相比之下，多测量了 KV cache 分配和清零、模型工作区内存分配消耗的时间
- 测量 TTFT
- 把**起点**设在 `llmrun_support.h` 中 `llm_drive_decode` 函数的 `LLM_LOG("starting forward pass\n");` 语句之后，**终点**设在 `for` 循环内 `gen_idx == 0` 的分支（对应第一个 token 生成的循环轮次）
- 测量 TPOT
- 对于每轮循环，把**起终点**分设在 `llmrun_support.h` 中 `llm_drive_decode` 函数的 `for` 循环内 `int next = token_forward(rt, kcache, vcache, linear_conv_cache, linear_state_cache, pos, ws);` 语句前后；在 `gen_idx == 0` 的否则分支中将单轮循环的计算结果累加到 `tpot_total` 变量中，并输出单轮循环的 token 生成用时
- 在循环结束后，利用 `double tpot = (double)tpot_total / total_count;` 语句，取平均值计算得出 TPOT

2. 测试数据记录表：

输入长度	输出长度	Model Loading Time (us)	TTFT (us)	TPOT (us)
8	8	2259443	17476034	2213623
8	16	2232256	17190833	2123948
8	32	2299628	17805992	2218257
8	128	2289159	16908962	2138732
16	8	2294955	35047067	2226477
16	16	2257970	34479389	2160836
16	32	2172756	33583974	2099692
32	8	2237370	68155390	2124890
32	16	2261376	68476008	2146982
32	32	2188746	67435579	2119661

其中 8/16 tokens 输入的测试**直接使用命令行**调用 `llmrun_smol --predict`，32 tokens 输入的测试使用 `llmrun_smol --stdin` 后在进程中输入具体参数

使用的测试命令：

测试编号	命令	输入	输出
1	<code>llmrun_smol --predict 8 1 2 3 4 5 6 7 8</code>	8	8
2	<code>llmrun_smol --predict 16 1 2 3 4 5 6 7 8</code>	8	16
3	<code>llmrun_smol --predict 32 1 2 3 4 5 6 7 8</code>	8	32
4	<code>llmrun_smol --predict 128 1 2 3 4 5 6 7 8</code>	8	128
5	<code>llmrun_smol --predict 8 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16</code>	16	8
6	<code>llmrun_smol --predict 16 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16</code>	16	16
7	<code>llmrun_smol --predict 32 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16</code>	16	32
8	<code>llmrun_smol --predict 8 32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1</code>	32	8
9	<code>llmrun_smol --predict 16 32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1</code>	32	16
10	<code>llmrun_smol --predict 32 32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1</code>	32	32

3.
 1. 哪些指标主要受输入长度影响？哪些指标主要受输出长度影响？
 - **TTFT** 主要受输入长度影响，**Model Loading Time** 基本不受输入/输出长度影响；**TPOT** 在输入规模较小时几乎不受输出长度的影响，但**理论上与输出长度成正相关**
 2. 哪些阶段反映系统的计算压力？哪些反映访存（内存带宽）压力？
 - **TTFT** 主要反映计算压力
 - **TTFT** 主要对应 Prefill 阶段的工作，需要对整段输入执行完整的层计算，因此主要受计算吞吐限制
 - **TPOT** 主要反映访存压力
 - **TPOT** 主要对应 Decode 阶段的工作，随生成 token 数增加，需要从 KV Cache 中读取的数据量逐渐增加，访存压力逐渐增大
 - 在**输出长度规模较小**（百级）时，读取的历史缓存主要取决于固定大小的模型权重，**TPOT** 随生成 token 数增加而增加的趋势极其微小而被每次运行时的其它偶然误差因素掩盖
 - **Model Loading Time** 主要包括从外存向内存写入造成的时间延迟，**对系统存储 I/O 造成的压力较大**，与系统计算压力关系不大

任务 B

1. 测试数据记录表：

输入长度	输出长度	Model Loading Time (us)		TTFT (us)		TPOT (us)	
		Qwen	Smol	Qwen	Smol	Qwen	Smol
4	4	12095779	2192168	46126192	8525436	11428424	2113945

输入长度	输出长度	Model Loading Time (us)		TTFT (us)		TPOT (us)	
		Qwen	Smol	Qwen	Smol	Qwen	Smol
	8	12134238	2183354	44883160	8600081	11161416	2151572
	16	11933865	2172106	44592737	8827543	11078618	2210261
	4	12298940	2188967	89551796	17310485	11133178	2311533
8	8	12189074	2259443	89423254	17476034	11076665	2213623
	16	12307713	2232256	90469045	17190833	11237100	2123948
	4	12090780	2164712	176908532	35681330	10969766	2246885
16	8	11852346	2294955	180541475	35047067	11262723	2226477
	16	12227243	2257970	178032419	34479389	11119402	2160836

- 在 Model Loading Time 的表现上，Qwen3.5-0.8B 的用时平均约为 SmolLM2-135M 的 5.47 倍；
- 在 TTFT 的表现上，随着输入的 token 数增加，Qwen3.5-0.8B 与 SmolLM2-135M 的用时比值缓慢减小 (4,tokens,5.23x → 8,tokens,5.19x → 16,tokens,5.08x)
 - 大模型的**固定开销**（权重加载、框架调度等）更大，但**计算效率**（矩阵乘法等）相比小模型有所优化

2. Qwen3.5-0.8B 和 SmolLM2-135M 的部分关键维度差异

关键维度	Qwen3.5-0.8B	SmolLM2-135M
模型整体参数量	~800M	~135M
Transformer 层数	24	30
隐藏层维度	1024	576
MLP 中间维度	3584	1536
注意力头数	8	9
词表大小	~248k	49152

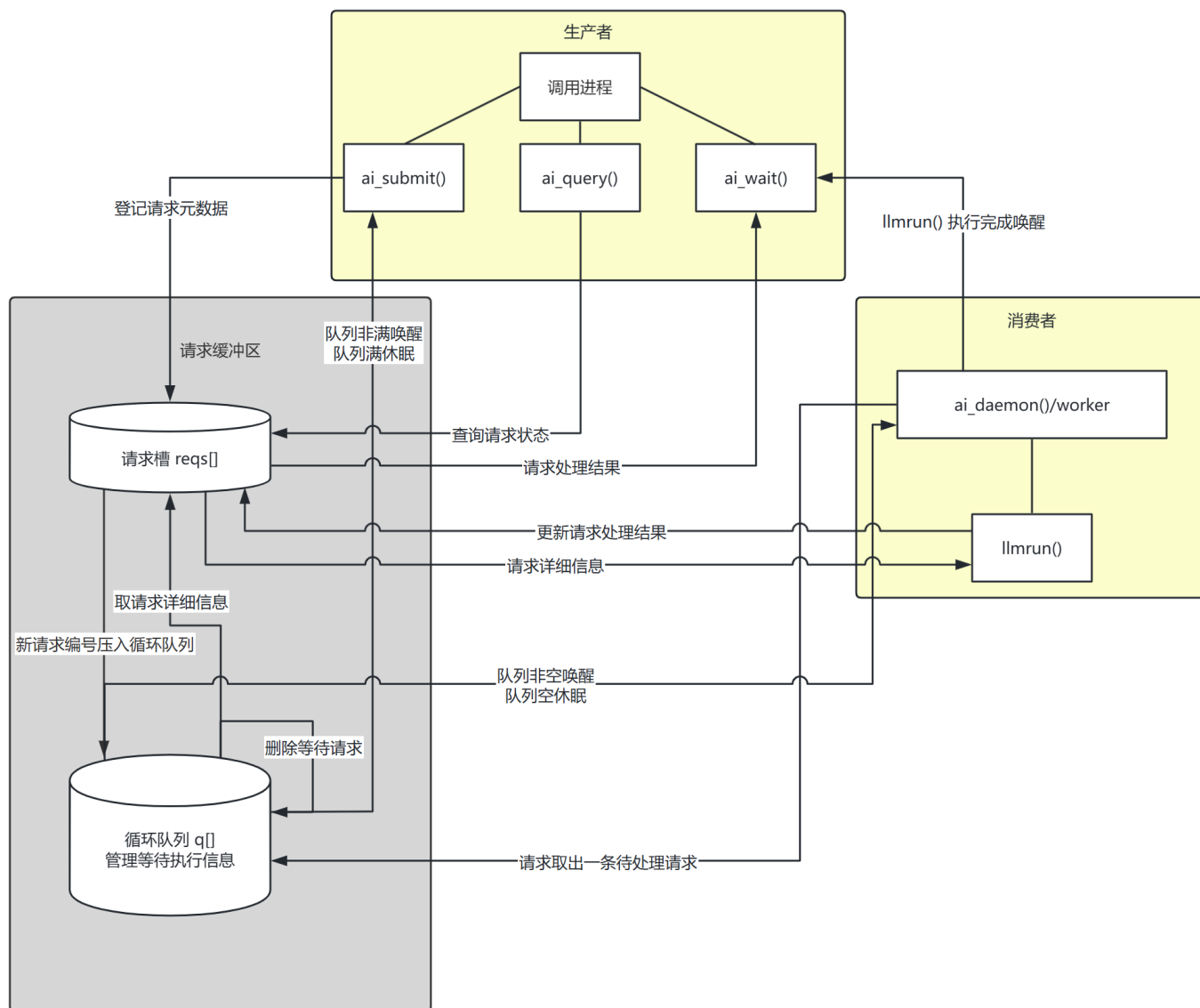
- Model Loading Time 的影响因素：总参数量
 - Qwen3.5-0.8B 的权重是 SmolLM2-135M 的约 5.9 倍，即两种模型加载时权重文件**向内存写入的总字节数存在 5.9 倍的关系**
 - 一些其它时间开销主要用于**权重格式解析、GPU 显存分配**等，这些开销都与模型总参数量大致成正比
- TTFT 的影响因素：前向计算量
 - Self-Attention 层上，Qwen3.5-0.8B 的 Q/K/V 投影矩阵为 1024×1024 ，而 SmolLM2-135M 为 576×576
 - MLP 层上，Qwen3.5-0.8B 的升维/降维矩阵为 1024×3584 ，而 SmolLM2-135M 为 576×1536

- 综合 Transformer 层数上的差异，可知 Qwen3.5-0.8B 的总矩阵运算算力需求仍然显著高于 Qwen3.5-0.8B
- TPOT 的影响因素：显存带宽
 - 在生成后续 token 时，需要将所有权重和历史 KV Cache 从显存读取到计算单元中
 - Qwen3.5-0.8B 的隐藏层维度和 MLP 中间维度显著高于 Smo1LM2-135M，意味着在每个 token 的产生过程中，Qwen3.5-0.8B 需要读取的权重切块、激活值规模更大，对显存带宽的压力更显著

Part 2

任务 A

绘制的异步服务结构图如下：



任务 B

1. 队列空，worker 无事可做

- 本质上是消费端等待生产端产出数据
- worker 检查到循环队列空时，进入睡眠状态；循环队列插入新请求编号时唤醒 worker
- 阻塞和唤醒参数
 - 阻塞者：后台 worker
 - 唤醒条件：循环队列插入新的待处理请求编号

- 唤醒者：循环队列

2. 槽位满，新的提交无法进入

- 本质上是**生产端等待消费端处理堆积的数据**
- `ai_submit()` 检查到请求槽满或循环队列满时，进入睡眠状态；循环队列删除一条待处理请求时唤醒 `ai_submit()`
- 阻塞和唤醒参数
 - 阻塞者：`ai_submit()`
 - 唤醒条件：循环队列有一条请求被 worker 接收并处理
 - 唤醒者：循环队列

任务 C

1. 为什么“把 AI 推理直接塞进一次同步调用”不是长期服务的好结构？

- 同步结构下，AI 推理调用者被完整的推理和输出流程**阻塞**
- 同步结构下，每次推理调用都会**付出模型装载和缓存分配成本**
- 同步结构**难以应对并发请求**带来的问题

2. 为什么“submit + 后台 worker + wait/query”更适合 AI 这类负载？

- 模型冷启动（模型装载和缓存分配等）成本较高，采用单一后台 worker 长驻可以**复用模型冷启动结果**
- 提高背压能力，在缓冲区待处理请求数量溢出时可以**阻塞生产端继续添加待处理请求**
- 遇到并发请求时，可以通过一个缓冲队列让这些请求“排队被处理”，**明确它们被处理的先后顺序**
- 对于生产端（调用者）而言，实现提交推理请求与等待推理结果返回的解耦，调用者**可以在等待推理结果返回之前继续执行其它任务**

Part 3

任务 A

1. 最小可行的 `ai_request` 元数据字段

- `reqid`：一个请求的唯一标识
- `owner_pid`：发起此请求的进程 PID
- `state`：存储请求的处理状态
 - 对于请求错误的情形，通过细分在 `state` 中定义的状态可以实现返回具体错误类型
 - 对于返回结果长度溢出的情形，可以**将返回结果长度检查前移到后台 worker 生成结果的过程中**，并通过 `ai_submit()` 在提交请求时同时指定缓冲区大小

2. 最小可行的 `ai_status` 对外可见字段

- `state`：存储请求的处理状态

任务 B

1. 为什么一个 AI 请求必须记录“属于谁”

- 只有**请求的主体**才允许通过 `ai_wait()` 等待请求完成并回收请求结果；为此需要在 `ai_wait()` 中引入权限验证模块
- 权限验证模块的功能需要**基于对请求主体的记录**

2. 为什么“知道 `reqid`”不等于“有权限 `wait/query`”

- 任何进程都可以通过**循环枚举**的方法取得一个请求的 `reqid`
- 只有请求的调用进程与请求之间存在父子关系，并具有 `wait/query` 的权限

3. 为什么一个请求完成后，结果被成功取走一次就应该回收，而不是永久保留
 - 避免请求在**有限容量**的请求槽中单调堆积，最终造成请求槽溢出无法接受新请求
 - 避免请求结果被重复取出，难以**将取出的结果与请求一一对应**，也难以**定义请求可以被回收的时间节点**，此外还造成**资源的二次释放**等安全隐患
 - 由于 **reqid** 范围有限，在一些极端高并发条件下，可能造成**新旧请求持有同一个 reqid** 的情形，导致整个请求数据结构的混乱

任务 C

1. 提交者在请求尚未完成时调用 **ai_query(reqid, st)**
 - **期望行为**
 - 调用成功返回
 - 在 **st.state** 中写入 **QUEUED/RUNNING** 等请求中间态
 - **理由**
 - 提交者具有对请求的查询权限，**ai_query()** 经权限验证后可以正常执行对目标请求的状态查询并返回
 - 请求尚未完成，在 **st.state** 中写入目标请求的当前处理情况
2. 提交者在请求完成后第一次调用 **ai_wait(reqid, out, cap)**
 - **期望行为**
 - 调用成功返回
 - 将请求处理结果写入 **out**
 - 销毁请求，回收请求槽
 - **理由**
 - 提交者具有对请求的查询权限，**ai_wait()** 经权限验证后可以正常执行对目标请求的结果提取并返回
 - 同一个请求在处理完成后只能提取一次，在第一次提取完成后应及时销毁
3. 同一个提交者第二次对同一 **reqid** 调用 **ai_wait(...)**
 - **期望行为**：调用失败
 - **理由**：成功提取一次请求结果后请求即被销毁
4. 其他进程或不具有所有权的子进程拿到 **reqid** 后调用 **ai_query(...)**
 - **期望行为**：调用失败
 - **理由**：在 **ai_query()** 中会预先对调用进程进行请求的权限验证，其他进程或不具有所有权的子进程即使获取到了请求的 **reqid**，也无法通过相应的权限验证